TreeSet class in collection

What is TreeSet :-

- The TreeSet class in Java is a part of the **java.util package**.
- It is implements the Set interface, as well as the **SortedSet** and **NavigableSet** interfaces.
- It is a collection that **stores unique elements** in a **sorted order.**

Hierarchy of TreeSet :-

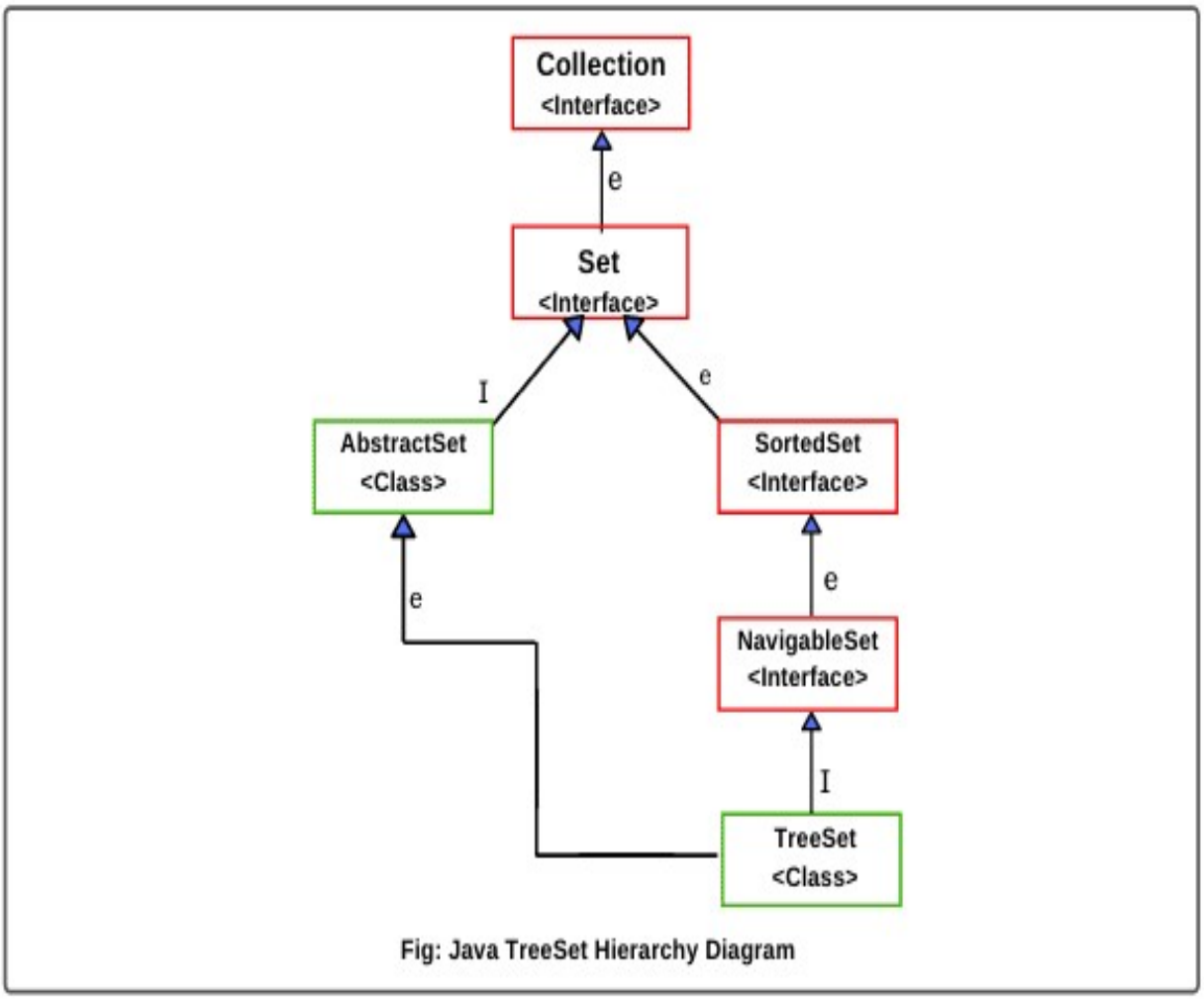

Fig: Java TreeSet Hierarchy Diagram

Key Points of TreeSet :-

- **Implements:-** NavigableSet, SortedSet, and Set interfaces.
- **Order:-** Maintains elements in **ascending (natural) order** automatically.
- **Duplicates:-** Each element must be unique.
- **Null Elements:-** Adding null causes a NullPointerException.
- **Underlying Data Structure:-** Uses a **Red-Black Tree** internally.
- **Synchronization:- Not synchronized** (you can make it synchronized manually using Collections.synchronizedSet()).
- **Performance:-** Operations like add(), remove(), and contains() take **O(log n)** time complexity.

Syntax of TreeSet :-

- **Generic :-**
  TreeSet<String> names = new TreeSet<String>();

- **Non-Generic :-**
  TreeSet names = new TreeSet();

Why Use TreeSet?

- When you need to store **unique elements**.
- When you want elements to be **automatically sorted**.
- When **searching** and **range queries** are frequent.

Methods in TreeSet :-

Method	Description
add(Object o)	Adds element in sorted order: ignores duplicates.
addAll (Collection c)	Adds all elements from a collection, ignores duplicates. Aote.
ceiling?(E e)	This method returns the Comparator used to sort elements in TreeSet or it will return null if the default natural sort...
clear()	This method returns true if a given element is
Comparator comprator ()	This method returns a reverse ordíer vewer of the e lteménts in méisels in descending order.
contains(Object0 o)	This method returns a revers order view of the elements contained in this set.
descendingl Iterator (()	Returns he firsteheñement in TreéSet if TreéSet or null elsa it mhríow NoSuchElementException
first(.e)	Returns the first element in TreeSet-d toss than or eual to it expiú elí no such element.
headSet (Object toElement)	This method returns elements of TreeSet that are less than specified element.
higher(E e)	This method returns the least element in thee Set if ñ trict lly greater than the given element, or null
isEmpty()	if no such element.

Difference between HashSet and TreeSet :-

Feature	HashSet	TreeSet
Order	No order maintained	Sorted order
Performance	Faster (O(1))	Slower (O(log n))
Null Values	Allows one null	Does not allow null
Implementation	Hash Table	Red-Black Tree

Difference between Linked HashSet and TreeSet :-

Feature	LinkedHashSet	TreeSet
Ordering	Maintains **insertion order**	Maintains **sorted (ascending)** order
Underlying Data Structure	Uses a **LinkedHashMap** internally.	Uses a **Red-Black Tree** internally.
Duplicates	Not allowed	Not allowed
Null Elements	Allows **one null element**.	Does **not allow null**
Performance	Faster for insertion.	Slower time for operations.
Sorting	Does **not sort** elements automatically.	**Automatically** sorts elements
Use Case	Maintain **insertion order** and don't need sorting.	There need elements in a **sorted order**.
Comparator Support	Not supported.	Supports custom sorting using Comparator.

Implementation of TreeSet :-

Input :-

```java
package Collection_Framework;

import java.util.Iterator;
import java.util.TreeSet;

public class TreeSet_generic {

    public static void main(String[] args) throws InterruptedException {

        TreeSet<String> sweets = new TreeSet<String>();
        sweets.add("Jalebi");
        sweets.add("Ladu");
        sweets.add("Gulabjam");
        sweets.add("Pedha");
        sweets.add("Barfi");

        System.out.println("Sweets are : "+sweets);

        System.out.println("Contains : "+sweets.contains("Pedha"));

        // descending set, iterator, clear, getfirst, getlast
        System.out.println("Size : "+sweets.size());
        System.out.println("Floor : "+sweets.floor("Gajar halwa"));
        System.out.println("First : "+sweets.first());
        System.out.println("Lower : "+sweets.lower("rasmalai"));
        System.out.println("Higher : "+sweets.higher("Badam Shake"));

        System.out.println("Pollfirst : "+sweets.pollFirst());
        System.out.println("After pollfirst sweets are : "+sweets);

        System.out.println("PollLast : "+sweets.pollLast());
        System.out.println("After PollLast sweets are : "+sweets);

        System.out.println("Subset : "+sweets.subSet("Gulabjam", "Ladu"));
        System.out.println("isEmpty : "+sweets.isEmpty());
        System.out.println("Tailset : "+sweets.tailSet("Gulabjam"));

        // Iteration
        System.out.println("\nIterating....sweets....:");
        Iterator i = sweets.iterator();

        while(i.hasNext())
        {
            System.out.println(i.next());
            Thread.sleep(4000);
        }
    }
}
```

Output :-

```
Sweets are : [Barfi, Gulabjam, Jalebi, Ladu, Pedha]
Contains : true
Size : 5
Floor : Barfi
First : Barfi
Lower : Pedha
Higher : Barfi
Pollfirst : Barfi
After pollfirst sweets are : [Gulabjam, Jalebi, Ladu, Pedha]
PollLast : Pedha
After PollLast sweets are : [Gulabjam, Jalebi, Ladu]
Subset : [Gulabjam, Jalebi]
isEmpty : false
Tailset : [Gulabjam, Jalebi, Ladu]

Iterating....sweets....:
Gulabjam
Jalebi
Ladu
```